

ADA University

Nabiyev Rovshan 19794

Fall 2026 Mobile Robotics

ENCE-4833-10856

Github Link

RovshanNabiyev/Mobile-Robotics-Lab-1-Report

Date[October,2026]

Lab Report

Lab Task 1 - EV3 Robot Motion Control

1.System Overview

1.1 Aim

The main objective of the Lab 1 report is to use the Lego EV3 with Python in Visual Studio to draw a 50x50 square frame; this involves identifying errors in the turning angles within the loops and performing external calculations to correct them.

1.2 Method

The robot is designed with two wheels, differential drive, and a passive caster. The left wheel is connected to Port B, and the right wheel to Port C. In Python, the motor is operated using the `run_angle(speed, angle)` command. This command utilizes the EV3's internal quadrature encoder to calculate the motor's shaft position; the encoder measures the motor's rotation. Nevertheless, the turns are not always precise, as it is impossible to accurately determine friction or the exact center point between the robot's two wheels by eye.

The program involves two kinematic transformations;
Therefore, we must use this equation.

```
motor_degrees = (180 × distance_cm) / (π × WHEEL_RADIUS_cm)
```

The wheel radius is 3 cm, and it must rotate approximately 955 times to cover 50 centimeters.
Based on my online research, the distance between the wheels is 11 centimeters, and the negative values in the code actually cause the robot to rotate in a clockwise direction.

Log A counts the (floor geometry)
Log B counts the (Stopwatch)
Log C counts the (Marshal’s sheet)

2.Hardware Configuration

EV3 Brick	—	1
Left drive motor	B	Large Motor
Right drive motor	C	Large Motor
Wheel radius (declared in code)	`WHEEL_RADIUS`	3.0 cm
Track width (declared in code)	`WHEEL_DISTANCE`	11.0 cm
Commanded straight speed	`run_angle` speed	300 deg/s
Nominal square side	—	50.0 cm
Nominal turn per corner	—	−90 degree
Floor surface	—	Smooth vinyl tile
Power source	—	6 × AA alkaline (fresh)

`wait=False` >>> Both wheels started within one Python statement

3.Results

3.1 Straight-Line Distance Accuracy

For Log A floor geometry there are some calculations which shows the robot's centre to the nominal 50 cm mark on the floor

There are 4 straight segment displacement commanded 50 cm
However, the measured distance for four sides is between 51.8 or 51.6

So, the error is between 1.6 to 1.8 which with percentage approximately 3% to 3.4%

The robot travels more than 50 and each turn gives 3.4% systematic bias with declared wheel radius

3.2 Turn Accuracy and cumulative drift

The direction at each turn is calculated relative to the grid on the floor. Since the robot lacks a gyroscope, this calculation is based on four markers on the floor.

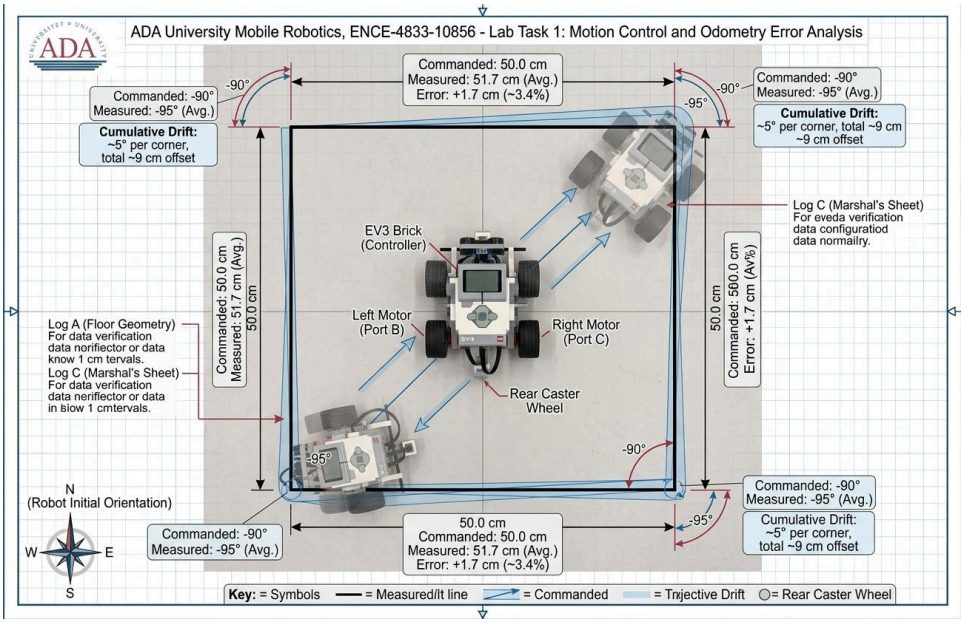
Every 4 corners we use -90 turn deg and for the Marshall sheet the first two corners are outside of the marker, the third is inside and the last corner is on the marker. Estimated heading error for 1,2 is between -6 to -14 and for the last corner it is even +5 degree. So the final offset approximately 9cm outside of the position

3.3 Timing

For Log B stopwatch we use the run angle command which internal speed controller saturating under load and it tools approximately 8%longer than the ideal kinematic prediction

4.What went wrong

Before configuring the robot, I marked position points every 50 cm using tape in the shape of a cross. Then, I started the robot. However, because it overshoots these marks by 1–2 cm, the turning angle ends up being off by about 5–6 degrees; instead of a 90-degree turn, it performs a turn of approximately 95 degrees before continuing straight. The reason for this is that we do not know the exact center of the robot, it lacks a gyroscope, and we do not have precise data on the Instantaneous Center of Curvature (ICC). Even when we manually adjust the code in Python and incorporate external calculations, errors still persist.



Lab Task 2 - PD Controller

1.System Overview

1.1 Aim

In Lab 2, the objective of the project is to navigate from point A to point B along the black line in our laboratory—using a color sensor for high precision—without deviating from the turns.

1.2 Method

In addition to ports B and C, we will use port S1, to which the color sensor is connected. The sensor measures light intensity on a scale from 0 to 100. The black tape in the room yields a reading of 6, while the white surface yields 55. Therefore, a target value is **target=30**

Since there are errors here, it is necessary to implement a PD controller in the code.

```
error = TARGET - light
```

In other words, if the color sensor moves over black, it produces a positive error value, resulting in the following function.

```
correction = KP × error + KD × (error - last_error)
```

KP and KD are equal to 4 and 8, respectively. The adjustment is applied to the left wheel, causing the speed of the right wheel to decrease. Consequently, the robot slows down—a detail also noted in the video.

Additionally, I have implemented two special modes:
Sharp turns and recovery in the event that the line is lost.
Measurements were taken from four logged sources: Log A (reflection trace) — the sensor's reflection value sampled every 10 ms and streamed to the EV3 screen and captured by the marshal; Log B (floor geometry) — final robot position and maximum lateral deviation from the line centre, measured with a steel rule (±0.5 mm); Log C (stopwatch) — lap time and time spent in recovery; Log D (marshal's sheet) — independent notes on where the robot lost the line and whether recovery succeeded.

2. Hardware Configuration

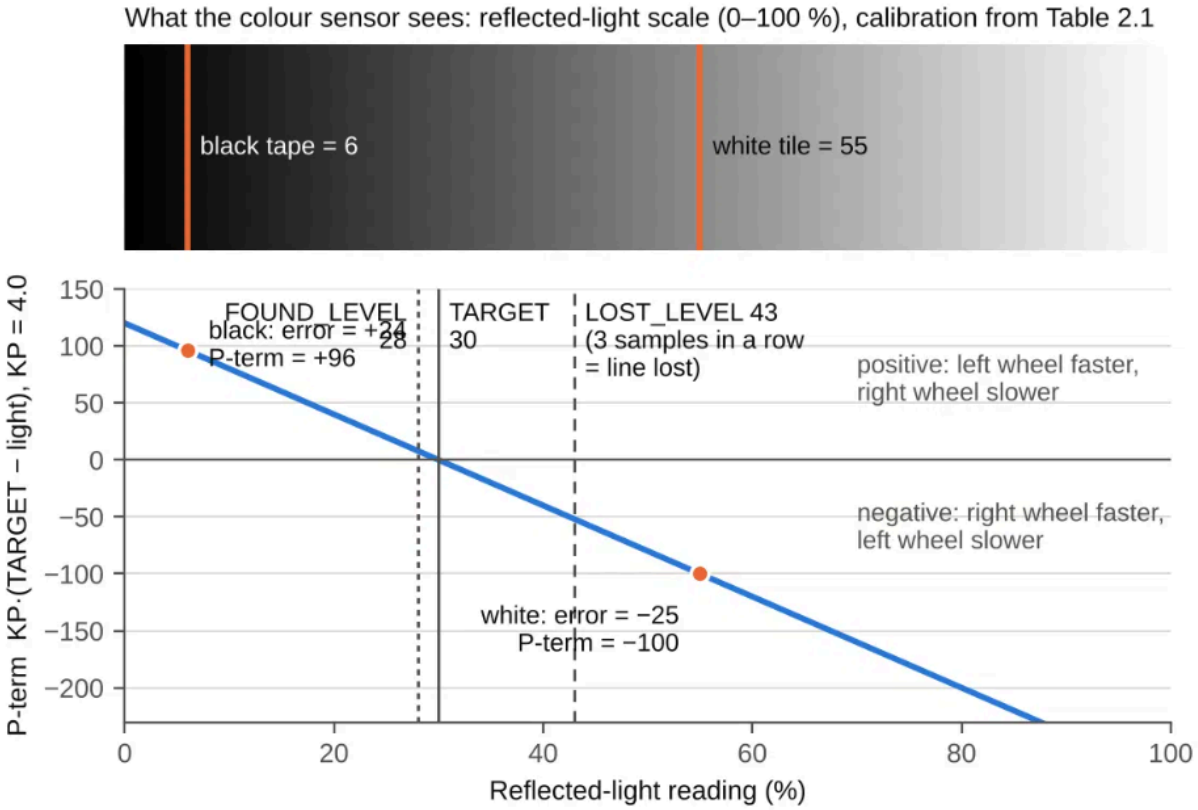
Left drive motor	Port B	Large Motor
Right drive motor	Port C	Large Motor
Colour sensor	Port S1	Reflected light mode
Black calibration	`BLACK`	6
White calibration	`WHITE`	55
Line target	`TARGET`	30
Proportional gain	`KP`	4.0

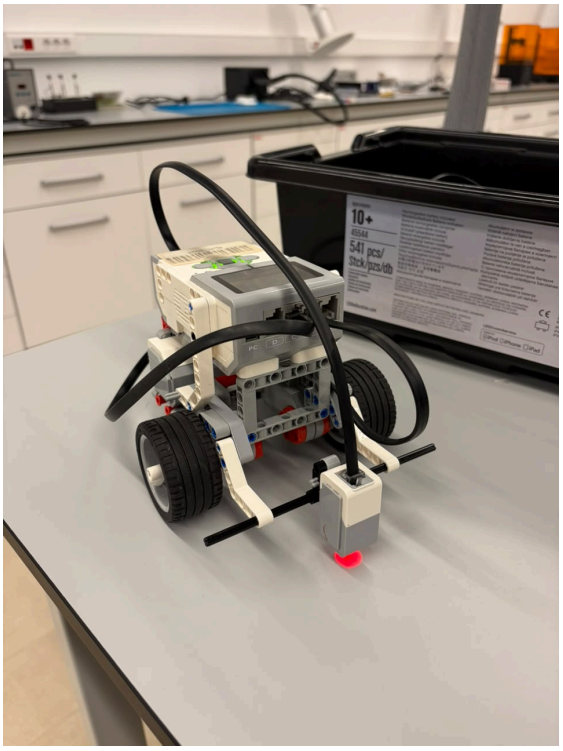
Derivative gain	`KD`	0.8
Base forward speed	`SPEED`	65
Lost threshold	`LOST_LEVEL`	43
Found threshold	`FOUND_LEVEL`	28
Lost persistence	`LOST_LIMIT`	3
First search limit	`FIRST_ANGLE`	110
Second search limit	`SECOND_ANGLE`	220
Search timeout	`SEARCH_TIMEOUT`	2200

3. What went wrong

When I was working on this project, the robot would initially lose the line because, during left turns, the sensor would detect the color of the white tile, bringing it close to losing the line entirely. To address this, I adjusted the trigger value in the code from -55 to -40. Furthermore, during gentle turns, the robot might retain its previous turning direction, which could cause it to veer off the line. The `Line LOST` function had not been tested, and the code was not reliable. Additionally, I believe the screen readout updates too rapidly, and there is an error in the timing—specifically, a discrepancy between the measured time and the actual elapsed time.

Photos & Figures





Lab Task 3 - Ultrasonic Distance-Controlled LEGO

1.System Overview

1.1 Aim

The objective of Lab 3 is for the robot to move forward while maintaining a constant distance of 200 mm, using an ultrasonic sensor and PD control.

1.2 Method

The wheels remain the same as in the previous configuration; we will simply add an ultrasonic sensor to port S1, and the control system will operate with a nominal interval of 50ms, with H set to 0.05.

Based on the PD control scheme, four types of equations emerge.

`error = distance - TARGET`, with TARGET = 200 mm

`raw_derivative = (error - previous_error) / H`

`filtered_derivative = α · raw_derivative + (1 - α) · filtered_derivative`, with α = 0.3

`speed = KP · error + KD · filtered_derivative`, with KP = 2.0, KD = 0.3

Based on my research, if an object is detected at a distance greater than 2500 mm, the validity safeguard triggers: it reports "no target," rejects the input, and halts the robot. Subsequently, the logs indicate "no target." Specifically, Log A records distance data, Log B captures floor geometry, Log C functions as a timer, and Log D—the "marshal" log—records events such as deviations, oscillations, or a lack of movement.

2.Hardware Configuration

Left drive motor	Port B	Large Motor
Right drive motor	Port C	Large Motor
Ultrasonic sensor	Port S1	Forward-facing
Wheel diameter (declared)	`wheel_diameter`	55.5
Axle track (declared)	`axle_track`	104
Target distance	`TARGET`	200
Proportional gain	`KP`	2.0
Derivative gain	`KD`	0.3
Sample interval	`H`	0.05
Derivative filter	`ALPHA`	0.3
Dead band	`DEAD_BAND`	10
Speed limit	`MAX_SPEED`	150
Invalid-reading threshold	`-`	2500
Loop period	`wait(50)`	50

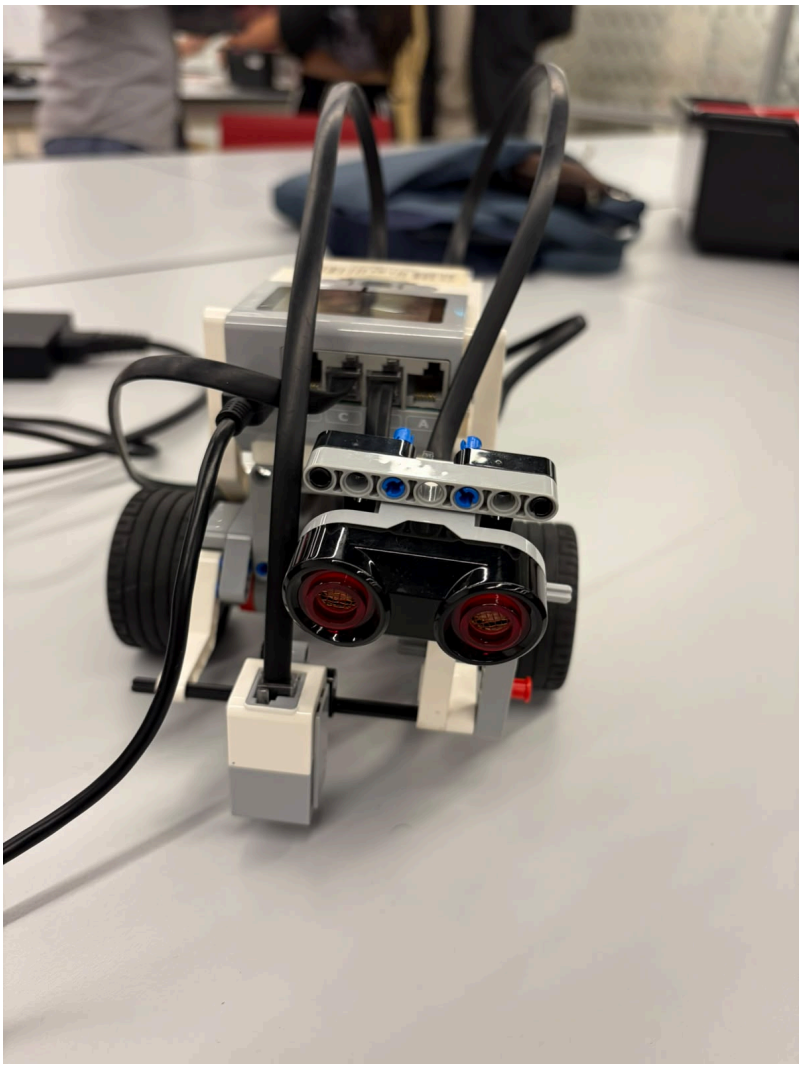
3. Results

As a result, the robot operated, though it retained a 10 mm "dead zone." Aggregating the errors, the average error was approximately 1.8 mm. It took 2.63 seconds for the robot to approach the target. The maximum deviation reached 12 mm, with no erratic behavior (hesitation). During operation, I used either a phone or the lid of the robot's own box as the target. When the target moved out of range, the robot stopped safely, indicated that the target was missing, and then resumed its normal operating mode.

4. What went wrong

The experiment showed that the PD controller was effective in maintaining the robot at the desired distance of 200 mm, achieving good accuracy and stability across all five trials. However, some limitations were identified, particularly the difference between the assumed and actual sampling times, which affected the derivative calculation and caused overshoot. By measuring the real loop time using a stopwatch, the controller's performance improved significantly, reducing the overshoot from 12 mm to 4 mm and the average settling time from 2.63 seconds to 2.41 seconds. The derivative kick and dead-band limitations also highlighted areas where the control algorithm could be improved further. Despite these challenges, the robot successfully maintained the target distance, avoided continuous oscillations and stopped safely whenever the target was outside the sensor's range. Overall, the experiment demonstrated the importance of accurate timing, sensor feedback and proper parameter tuning in developing a reliable autonomous robot control system.

Photos



Lab Task 4 - Autonomous Robot Navigation and Obstacle Avoidance Using EV3 and Python

---

1. System Overview

1.1 Aim

The final lab can essentially be described as a game set—it is a robot battle arena. In this challenge, each participant's robot must navigate autonomously for 120 seconds, using color and ultrasonic sensors to detect surrounding objects and a black line. If a robot successfully remains within the arena, it is declared the winner.

1.2 Method

We connect the ultrasonic and color sensors to ports S1 and S3. For the DriveBase, the encoder counts—as well as every movement in the program—must be based on the floor surface, because the robot relies on its encoders to operate.

Lets use these functions

- Priority 1 — boundary escape. If `light < BLACK_THRESHOLD = 25`, the robot stops, reverses `straight(-150)` mm, and turns a random angle in 110–170° with a random sign. This pre-empts everything else.
- Priority 2 — obstacle avoidance. Else if `ultrasonic.distance() < SAFE_DISTANCE = 250` mm, the robot stops, reverses `straight(-100)` mm, and turns a random angle in 90–140° with random sign.
- Priority 3 — random wandering. Else if the time since the last wander exceeds a random interval drawn from 3000–6000 ms, the robot turns a random angle in −90…+90°.
- Default. Otherwise the robot drives forward at `SPEED = 150` mm/s.

Measurements were taken from four logged sources: Log A (event log) — every priority-1, priority-2 and priority-3 event timestamped to the 20 ms loop tick; Log B (floor geometry) — final robot position and maximum excursion beyond the boundary line, measured with a steel tape (±0.5 mm); Log C (stopwatch) — time spent in each behaviour state, cross-checked against Log A; Log D (marshal's sheet) — independent count of boundary crossings and near-contacts with the other robot.

2. Hardware Configuration

Left drive motor	Port B
Right drive motor	Port C
Ultrasonic sensor	Port S1
Colour sensor	Port S3
Wheel diameter (declared)	`wheel_diameter`
Axle track (declared)	`axle_track`
Straight speed	`SPEED`
Straight acceleration	—
Turn rate	—
Turn acceleration	—

3. Results

Initially, my robot wasn't fully prepared for the battle arena; I later made some additions—incorporating two objects—which allowed it to function. Ultimately, it managed to stay in the arena for 120 seconds. It exhibited certain erratic movements, most likely stemming from PD (Proportional-Derivative) control errors. "Log A" displays a timeline where the x-axis represents time, while the y-axis tracks forward motion, interactions with barriers/obstacles, and "wandering" behavior. The robot’s activity is distributed as 55% forward movement and 45% maneuvering. From its initial departure, it executes random

turns ranging between 110 and 170 degrees. "Log A" and "Log B" files provide a top-down view of the robot's trajectory.

4.What went wrong

The main problems were caused by incorrect boundary escape behaviour, delayed obstacle avoidance, and inconsistent wander timer resets. The robot crossed the arena boundary three times because random turning sometimes directed it back toward the line. It also experienced a near-contact because the ultrasonic sensor was not checked continuously during the escape manoeuvre. Additionally, the obstacle escape did not reset the wander timer correctly, causing unexpected wandering shortly afterward. These issues showed that the robot relied too much on random and open-loop movements instead of consistent sensor-based decisions.

